Trace-Driven Bug Reproduction and Patching for Python

Project specification v1.0 (frozen). Prepared October 20, 2026.

1. Summary

This project builds and evaluates an automated pipeline that takes a production-style crash report (a stack trace plus exception data) for a Python library, reproduces the crash as a failing pytest test, and proposes a minimal patch. The primary deliverable is **not** the pipeline itself. It is a reproducible benchmark that measures how well the pipeline works, with honest statistics and a failure analysis.

Research question: Given only a runtime stack trace and static source context, how often can an LLM agent (a) write a test that faithfully reproduces the bug and (b) produce a patch that the maintainers' own regression test accepts?

Hypotheses (exploratory, stated before any results):

- **H1:** Writing and verifying a repro test before patching improves patch correctness compared with patching directly, at equal compute.
- **H2:** Most failures come from missing context (fault localization, missing external state), not from syntax or tooling errors.
- **H3:** Performance is lower on bugs fixed after the model's training cutoff than on bugs fixed before it, which would indicate some recall effect.

A null or negative result on any hypothesis is an acceptable outcome and will be reported as found.

2. Goals, Non-Goals, Definition of Done

What the project should demonstrate to a reviewer: systems engineering (sandboxing, harnesses), experimental rigor (oracles, statistics), security thinking (threat model with tested containment), and clear technical writing.

Non-goals (listed in Future Work, not built): eBPF or process-level tracing, database or external state reconstruction, languages other than Python, GitLab or Bitbucket, a hosted SaaS, a dashboard UI, model fine-tuning.

Definition of done, in three tiers:

Tier	Contents
Minimum (must ship)	Harness plus at least 20 gold-validated bugs; Pipeline A and B on one model; results table with confidence intervals; failure taxonomy; README, SECURITY.md, LIMITATIONS.md
Target	30 bugs; two models; 3 trials per bug; McNemar analysis; contamination split; recorded draft-PR demo
Stretch	Public write-up or blog post; a third model; extra repositories; a results web page

3. Positioning and Related Work

Be upfront in the README: this is **not a novel technology**. The contribution is a small, careful, reproducible study of a specific setting (stack-trace-driven, repro-first) with an oracle that checks repro fidelity.

Work	What it does	Relation to this project
Sentry Seer Autofix	Industry tool: root-cause analysis from Sentry telemetry, opens fix PRs	The commercial reference point; closed and hosted, no benchmark of this form
SWE-bench	Patch generation from GitHub issue text on full repositories	Closest benchmark methodology; this project uses traces, not issue text
SWT-bench	Test generation from issue descriptions	Related to the repro stage
BugsInPy	Curated dataset of real Python bugs with reproducing tests	Possible source of candidate bugs
LIBRO	LLM-based bug reproduction from bug reports (evaluated on Java)	Closest prior work on the repro-first idea

Before publishing, read each paper's abstract and method section and verify these descriptions. Cite properly.

4. System Architecture

The core engine has two front doors: **benchmark mode** (primary, reads bug manifests) and **live mode** (demo, reads a Sentry-style webhook).

```
Manifest or webhook  ->  Payload builder  ->  Context assembler
                                                    |
                                  Sandbox (docker, --net=none)
                                                    |
              Pipeline A: trace -> repro test -> verify fails -> patch
              Pipeline B: trace -> patch (with budget-matched retries)
                                                    |
                        Host-side patch validator (deny-list)
                                                    |
                   Fresh-container evaluation (oracles)
                                                    |
                   Metrics, taxonomy  |  Draft PR (live mode only)
```

Component	Technology	Responsibility
Dataset builder	Python, Git, Docker SDK	Builds one pinned image per bug, runs gold-patch preflight
Payload builder	Python	Creates the synthetic Sentry-style payload from a gold run
Context assembler	Python ast	Selects source files given to the agent
Agent	Thin LLM client interface	Repro and patch generation, prompts versioned in the repo
Patch validator	Python	Host-side checks before any patch is executed
Sandbox and evaluator	Docker SDK	Offline, locked-down execution and oracle runs
Analysis	pandas, scipy	Tables, confidence intervals, tests, figures
Live mode	FastAPI, GitHub REST	Webhook receiver and draft-PR publisher (last phase)

5. Dataset

5.1 Candidate pool

Pure-Python libraries only: click, requests, flask, jinja2, attrs, marshmallow, rich, werkzeug, starlette, httpx, typer, sqlparse. Backups if yield is low: tomlkit, itsdangerous, markupsafe, more-itertools.

5.2 Selection criteria (written and frozen before screening)

A bug is a candidate only if all of the following hold:

1. The fix commit has a regression test and the bug is a crash (an uncaught exception with a stack trace).
2. The parent commit builds in `python:3.11-slim` with pinned dependencies, and the test suite runs offline.
3. The gold fix touches only source files, with no deny-listed non-test files, in at most 3 files and about 40 changed lines.
4. The failing behavior is deterministic across repeated runs.
5. Fix date is recorded. Prefer fixes from the last 12 months.

5.3 Screening log

Keep a spreadsheet or JSONL of **every** candidate examined, with the reason for each exclusion. Expect to screen more than 100 candidates to reach 30. This log is evidence against cherry-picking and goes in the repo.

5.4 Gold-patch preflight (host checks, all must pass)

1. **Gold test:** the upstream regression test fails on the parent (buggy) commit with the target exception and passes on the fix commit.
2. **No gold regressions:** no test that passes on the buggy commit fails on the fix commit.
3. **Stability:** the baseline suite run 3 times on the buggy commit gives identical pass/fail results. Flaky tests are recorded and excluded from the baseline set.
4. **Offline isolation:** the image builds with network on, then runs pytest with `--net=none`.

Bugs failing any check are **dataset exclusions**, not model failures, and are never counted in a denominator.

5.5 Manifest fields (one file per bug)

Field	Meaning
id, repo, issue_url	Identifiers and provenance (the issue text is never shown to the agent)
c_bug, c_fix	Parent commit and human fix commit hashes
fix_date	Used for contamination cohorts
target_exception, target_frame	Exception type and innermost library frame (file and function)
payload_path	Synthetic Sentry-style JSON
oracle_tests	Test ids from the fix commit used as the held-out oracle
baseline_pass_set, flaky_set	Stable passing tests and excluded flaky tests
gold_files	Files touched by the human fix (used to compute fault-localization misses)
image_tag, python_version, deps_lock	Reproducible environment
notes	Curation remarks

5.6 Synthetic payload generation

Run the gold oracle test on the buggy commit with locals captured, then format the trace, exception type and message, and truncated frame variables in a Sentry-like JSON schema. State plainly in the README that payloads are synthetic. Issue text, commit messages, and PR descriptions never reach the agent.

5.7 Splits and cohorts

- **Dev set (5 to 6 bugs):** used only to iterate on harness and prompts. Never reused in the final set.
- **Test set (target 30, minimum 20):** frozen before the final run.
- **Contamination cohorts:** tag each bug as pre-cutoff or post-cutoff per model using the fix date and the vendor's stated training cutoff. Cohort analysis is descriptive only, since each cohort will be small.

6. Agent Design

Context given to the agent (static, no browsing tools): the source files containing trace frames, plus source files directly imported by them (1-hop), plus the synthetic payload. Apply a fixed token cap with a documented truncation rule. No network, no repository search, no access to tests beyond what the context includes.

Pipeline A (repro-first):

1. Generate a repro test from the payload and context (up to 2 attempts).
2. Agent-time gate: the test must fail on the buggy code with the target exception type from the target frame.
3. Generate a patch (up to 2 attempts, using the repro and the existing visible tests as feedback).
4. Submit for evaluation.

If no valid repro is produced after the attempts, the run stops and is recorded as a No-Repro failure. This strict rule is the primary analysis. If time allows, also report a secondary variant that falls back to direct patching.

Pipeline B (direct patch): generate a patch, with up to 2 iterations using the existing visible test suite as feedback, then submit for evaluation.

Budget matching: both pipelines share the same per-run cap on total input and output tokens. Repro generation counts against Pipeline A's budget. Report cost per arm next to accuracy so the comparison is transparent.

Configuration discipline:

- Prompts are stored in the repo and versioned. Freeze them (git tag) before the final test-set run.
- Use two current models from different vendors (for example Claude Sonnet 5.5 and the current OpenAI flagship). Record exact model identifiers and run dates, because APIs retire models.
- Wrap providers in one client interface and **log every raw request and response**. All analysis must be reproducible from logs without re-spending money.
- Patch format is a unified diff with limits on files and changed lines (default 3 files, 40 lines, reported in the paper).

7. Evaluation Harness

7.1 Host-side patch validator (runs before anything executes)

Reject a patch if it:

- contains absolute paths, `../`, symlinks, or binary content;
- **modifies or creates** any test file (`tests/*`, `test_*.py`, `*_test.py`);
- modifies or creates test or interpreter configuration (`conftest.py` at any depth, `pytest.ini`, `tox.ini`, `setup.cfg`, `sitecustomize.py`, `usercustomize.py`, `.pth` files);
- modifies dependency or CI files (`pyproject.toml`, `setup.py`, `requirements*.txt`, `.github/*`);
- exceeds the diff size limits.

7.2 Sandbox configuration

Run with: `--net=none`, `--read-only`, tmpfs for `/tmp` and cache directories, a non-root user, `--cap-drop=ALL`, `--security-opt no-new-privileges`, `--pids-limit`, CPU and memory limits, a hard wall-clock timeout, no Docker socket, `PYTHONDONTWRITEBYTECODE=1`, and `-p no:cacheprovider`. Mount the repository as a writable copy or overlay where the patch is applied, since the root filesystem is read-only.

7.3 Separation of agent time and evaluation time

The agent never sees `C_fix` or the oracle tests. Evaluation runs in a **fresh container**: oracle test files are copied in from `C_fix`, and the patch is re-applied from the host. Nothing the agent's container produced is trusted.

7.4 Oracles

Repro oracle (a generated test is valid only if all three hold):

1. It fails on `C_bug`.
2. The failure is the target exception type, raised from the target frame (file and function, not an exact line).
3. It passes on `C_fix` without modification.

Patch oracle (a patch is correct only if both hold):

1. The held-out oracle tests from `C_fix` pass on `C_bug` plus the patch.
2. No test in the stable baseline set fails.

7.5 Manual audit

Oracle tests can over-specify and reject valid alternative fixes. Manually inspect **every patch judged correct** (to catch tricks such as special-casing pytest in source code) and a random sample of about 25% of patches judged incorrect, to estimate the oracle's false-negative rate. Report that rate.

8. Metrics and Failure Taxonomy

Metrics (all with raw counts and 95% Clopper-Pearson intervals where proportions): repro rate, patch accuracy, pass@3 (any of 3 trials succeeds), consistency (all 3 succeed), cost per run (tokens and dollars), wall-clock time, diff size.

Failure taxonomy. Each failed run gets exactly one label, assigned by this precedence (top first):

#	Category	Definition
1	Scope Violation	Patch rejected by the host validator
2	Syntax or Import Error	Patch or test does not parse or import
3	No-Repro	Generated test passes on `C_bug` (Pipeline A only)
4	Invalid Repro	Test fails on `C_bug` for the wrong exception or frame, or fails on `C_fix` (Pipeline A only)
5	Fault Localization Miss	Gold diff touches files outside the provided context (computed automatically from `gold_files`)
6	State Deficit	Bug needs external state absent from the payload (labeled by hand, blind to results where possible, second reviewer on a sample)
7	Regression Failure	Patch breaks a baseline test
8	Incorrect Patch	Patch applies and passes the baseline but fails the oracle tests

Categories 3 and 4 do not apply to Pipeline B, so state how the two arms are compared (patch accuracy and categories 1, 2, and 5 to 8).

9. Statistical Analysis Plan

Commit this plan (git tag) before the final run.

- **Unit of analysis is the bug, not the trial.** Collapse trials first: a bug counts as solved if at least 2 of 3 trials succeed.
- **Primary comparison:** Pipeline A versus B on patch accuracy, per model, using an exact McNemar test on the per-bug binary outcomes. Also report the paired difference in per-bug success rates with a bootstrap confidence interval.

- **Treat all analyses as exploratory.** Report effect sizes and intervals. Do not claim significance across multiple comparisons.
- **Power:** with N=30, only large differences are detectable. Say so in the write-up.
- **Contamination cohorts:** descriptive only, no test.
- **Cost:** a single run is roughly 5 model calls with 10k to 30k tokens of context each. With 30 bugs, 3 trials, 2 pipelines, and 2 models, that is 360 runs and tens of millions of input tokens. Measure real cost in Phase 2, then set a hard spending cap in the harness.

10. Security and Threat Model

Core assumption: indirect prompt injection through error messages and payloads **will succeed**. Delimiters such as `<trace>` tags and regex scrubbing reduce accidental confusion but are not a security boundary, and SECURITY.md must say so.

Containment layers:

- Execution: offline container, non-root, dropped capabilities, resource and time limits, no secrets in the environment, read-only filesystem.
- Patch scope: host-side deny-list (section 7.1) blocks workflow, dependency, and test-config changes, which are the classic supply-chain targets.
- Publishing: the GitHub token exists only in the PR module on the host, scoped to a **personal fork or a throwaway demo repository**. All PRs are drafts requiring human review. **Never open automated PRs against real upstream projects.**
- Host hygiene: run the benchmark on a disposable VM or a machine without personal secrets, since container isolation reduces but does not eliminate escape risk.

Show it, do not just claim it. Include a red-team test in `tests/`: a payload whose exception message contains injection instructions and a patch that attempts network access, secret reads, and writes to a workflow file. The test asserts that each attempt is contained or rejected. This is a strong portfolio artifact.

11. Engineering Quality

```
benchmark/   dataset manifests, screening log, run configs, results (JSONL
logs)
src/         dataset/, payload/, context/, agent/, sandbox/, evaluate/,
analysis/, live/
prompts/     versioned prompt files
tests/       harness meta-tests, red-team tests
docs/        ARCHITECTURE.md, METHODOLOGY.md, LIMITATIONS.md, SECURITY.md
Makefile     build-images, validate-gold, run-benchmark, analyze
```

- **Harness meta-tests with known answers:** the gold patch must pass, an empty patch must fail, a patch editing `conftest.py` must be rejected, and a test-skipping patch must be rejected. This proves the benchmark measures what it claims.
- **Reproducibility:** pinned dependencies, one YAML config per run, structured JSONL logs, and `make analyze` regenerating every table and figure from stored logs without API keys.
- **Quality gates:** ruff and pytest in GitHub Actions (for the harness, not for LLM runs), type hints on public functions.
- **Licensing:** MIT for your code. Manifests reference upstream commits instead of copying code, and upstream licenses are respected.

12. Portfolio Packaging

README structure: one-paragraph pitch with the headline result, results table with confidence intervals, architecture diagram, quickstart (reproduce tables from cached logs in under 5 minutes), methodology summary, failure analysis, limitations, security summary, future work.

Figures: pipeline diagram; results table; failure taxonomy bar chart per pipeline; per-bug pass/fail heatmap by pipeline and model; cost versus accuracy plot.

Case studies (4 total): two successes and two failures, each showing the actual payload, the generated repro, the patch, and the verdict. Failure case studies are often more convincing than successes.

Write-up: a 1,500 to 2,500 word post covering the question, method, results, surprises, and limitations. Publish results even if they are weak.

Demo: a 2 to 3 minute screen recording of live mode: webhook in, draft PR out on your own fork.

Resume bullet template (fill in real numbers): Built and evaluated an LLM pipeline that reproduces Python crashes from stack traces and proposes patches; benchmarked on [N] historical bugs from [K] libraries with a dual-oracle harness, reaching [X/N] valid repros and [Y/N] correct patches (95% CI [a, b]); found [headline finding from the ablation]; designed a sandbox and threat model with a tested containment layer.

Interview talking points: why you chose a benchmark over a product, what the dual oracle catches, what a confidence interval at N=30 means, the most surprising failure, what you would do with more time, and why prompt injection is handled by containment instead of filtering.

13. Timeline and Scope Control

Assumes about 12 weeks at part-time pace. Scale to your real deadline.

Phase	Weeks	Deliverable	Exit criteria
0. Setup and freeze	1	Repo, this spec tagged, candidate list, frozen selection criteria	Spec tagged v1.0
1. Harness and dev bugs	2 to 4	Validator, sandbox, evaluator, 5 dev bugs	On all 5: gold patch passes, empty patch fails, tampering rejected
2. Pipelines	4 to 6	Pipeline A and B, logging, cost measurement	Both run end to end on dev set; spending cap set
3. Dataset scale-up	5 to 8	Screening log, validated manifests	30 manifests (at least 20) frozen
4. Freeze and run	8 to 10	Prompts tagged, full runs	All logs stored
5. Analysis	10 to 11	Tables, tests, taxonomy labeling, audit	Results reproducible via make analyze
6. Demo and writing	11 to 12	Webhook, draft PR on own fork, README, write-up	Clean-clone quickstart works

Scope-cut ladder (apply in this order if behind): reduce N from 30 to 25, then to 20; reduce trials from 3 to 2; drop the second model; replace the live demo with a recorded mock; drop the contamination analysis. **Never cut:** gold validation, oracle checks, tamper protections, the audit, and honest limitations.

Risk	Mitigation
Low dataset yield	Backup repositories; relax the date preference first, not the oracle checks
Environment build failures	Exclude at curation; budget time per bug
Cost overrun	Spending cap; develop on a cheaper model; cache everything
Oracle false negatives	Manual audit and reported rate
Model or API changes	Pin identifiers; cache raw logs

Risk	Mitigation
Time overrun	Scope-cut ladder

14. Business Validation Track (Parallel, Light)

Start around week 6, once early results exist. The goal is a cheap answer to whether anyone would pay, not a second project.

- Hold about 10 conversations with engineers or team leads (classmates with internships, online communities, local meetups). Ask how they handle production bugs today, where time goes (detection, triage, reproduction, fix, review, deploy), and what they already pay for. Listen; do not pitch.
- **Signals worth following:** reproduction or triage described as costly, a need for self-hosted or air-gapped tooling, stacks without Sentry, or a request for a pilot.
- **Kill criterion:** if after 10 conversations nobody describes this as a real pain, treat the project as portfolio-only. That is a fine outcome.
- **Realistic early revenue paths:** integration consulting, a repro-only tool, or a self-hosted offering for restricted environments. Head-on competition with hosted platforms is a hard road for a solo developer, and trust (repo write access for an unknown vendor) is the main barrier.

15. Frozen Decisions and Change Rule

Frozen: Python and pytest only; Docker isolation; the 12-repository pool; target N=30 with minimum 20; 3 trials; two pipelines; two models; the 3-condition repro oracle and held-out patch oracle; the strict Pipeline A rule; the deny-list; the analysis plan.

Change rule: the spec changes only through a logged amendment (date, reason, impact), and only for (a) a bug in the harness or (b) infeasibility discovered by running something. "It would be nicer" is not a valid reason. This rule is how the project avoids endless redesign.

16. Decisions Only You Can Make

- Which two models, and your total API budget.
- Your real deadline, to scale the timeline.
- Whether the repository is public (recommended) and where the write-up is published.
- The final repository list after early screening.

First Three Actions

1. Create the repository, add this spec, and tag it v1.0.
2. Write the selection criteria file and start the screening log with 10 candidate bugs from click, requests, and flask.
3. Take one candidate through the full gold-patch preflight by hand. Whatever breaks teaches you what the harness needs.